```
for(int i=0; i<strlen(text); i++) {
    if(text[i] == '\n')
      break;
    hamming_encode(text[i], code);
    for(int j=1; j<=12; j++) {
      fprintf(fp, "%d", code[j]);
    fprintf(fp, " ");
  }
  fclose(fp);
  printf("Data encoded successfully");
  return 0;
}
```

Code : (receiver).

```
#include <stdio.h>
int main(){
    file *fp;
    char block[20];
    int code[13];
    int data;
    char ch;
    int error_position;
    fp=fopen("channel.txt","r");
    if(fp==NULL){
    printf("Error");
    return 1;
    }
    printf("\n Received data:\n");
    while(fscanf(fp,"%s",block)==1){
      for(int i=1; i<=12; i++){
          code[i]=block[i-1]-'0';
      }
      error_position = p1*1 + p2*2 + p4*4 + p8*8;
```

```
if (error_position == 0){
  printf ("No errors detected");
}
else {
  code [error_position] = code [error_position] ^ 1;
  printf ("Error corrected");
}
data = 0;
data = (data << 1) | code [3];
data = (data << 1) | code [5];
data = (data << 1) | code [6];
data = (data << 1) | code [7];
data = (data << 1) | code [9];
data = (data << 1) | code [10];
data = (data << 1) | code [11];
data = (data << 1) | code [12];
  ch = (char) data;
  printf (" Received character: %c \n", ch);
}
  fclose (fp);
  printf ("Data received successfully. \n");
  return 0;
}
```

Output:

Enter the text: Hello

Data encoded successfully.

Hamming Coded data is stored in channel.txt.

Scanned with OKEN Scanner

EX:NO:7 Flow control at Data link layer.

Aim:

Write a program to implement flow control at data link layer using sliding window protocol.

Create a sender program:

i) Input window size from the user.

ii) Input a text message from the user.

iii) Consider 1 character per frame.

iv) Create a frame with following fields.

v) Send the frames.

vi) Wait for the acknowledgement from the receiver.

vii) Reader a file called receiver-buffer.

viii) Check ACK field for the acknowledgement number.

Create a receive file:

i) Reader a file called sender-buffer.

ii) check the frame no.

iii) If the frame no are as expected, write the approxiate ACK no in the receiver-buffer file.

Scanned with OKEN Scanner

code;

Sender program;

```
#include <stdio.h>
int main() {
  int window, i, start = 0, ack;
  char msg[100];
  file* fp;
  printf("Enter window size: ");
  scanf("%d", &window);
  int n = strlen(msg);
  while (start < n) {
    fp = fopen("Sender-buffer.txt", "w");
    printf("Sending frames ":);
    for(int i = start; i < start + window && i < n; i++) {
      printf("Frame No: %d, Data: %c\n", i+1, msg[i]);
    }
    fclose(fp);
    fp = fopen("Receiver-buffer.txt", "r");
    if (fp == NULL) {
    printf("Receiver-buffer.txt not found\n");
    continue;
    }
    fscanf(fp, "%d", &ack);
    fclose(fp);
    if (ack == start + window || ack >= n) {
      printf("ACK %d received", ack);
      start += window;
    }
    else {
      printf("Resending frames from frame %d:\n",
                                    start+1);
      }
    }
    return 0;
}
```

Scanned with OKEN Scanner

Receiver program:

```
#include <stdio.h>
#include <unistd.h>
int main() {
    FILE *fp, *ackfp;
    char line[100];
    int frame, expected = 1;
    char data;
    while (1) {
     fp = fopen ("sender_buffer", "r");
     if (fp==NULL)
    {
     printf ("Waiting for sender");
      sleep (2);
      continue;
    }
    while (fgets (line, sizeof(line), fp)) {
    ackfp = fopen ("Receiver buffer", "w");
     if (frame == expected) {
       printf ("Frame %d accepted\n", frame);
        expected ++;
     }
    else {
        printf ("Frame %d rejected", frame);
        }
        fclose (ackfp);
      }
     fclose (fp);
     sleep (3);
  }
   return 0;
  }
```

Ex.No:8 NMAP to Discover live hosts using Nmap scans on the TryHackMe Platform.

Aim:

To discover live hosts using Nmap scans (ARP, ICMP, TCP/UDP) on the TryHackMe platform Room.

Introduction:

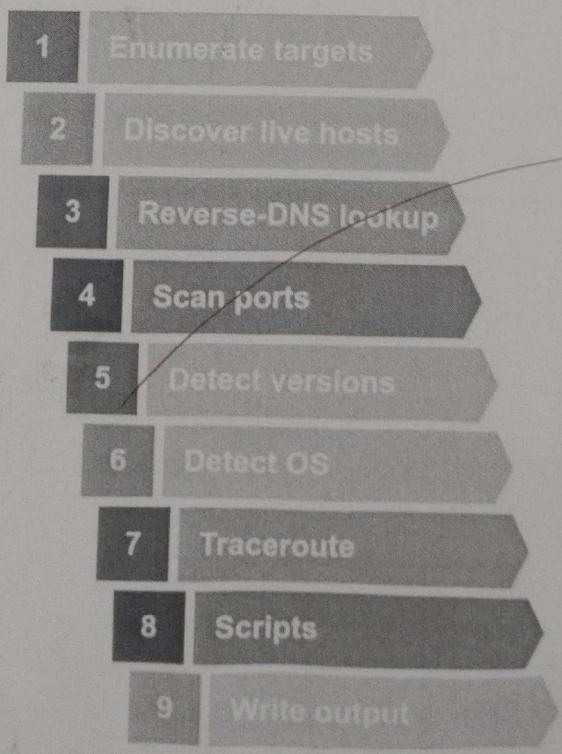
This room is the first of four in this Nmap series. These four rooms are also part of the Network Security module.

⇒ Nmap live host discovery.
⇒ Nmap basic port scans.
⇒ Nmap advanced Port scans.
⇒ NMAP post port scans.

ARP scan: This scan uses ARP requests to discover live hosts.

ICMP scan: This scan uses ICMP requests to identify live hosts.

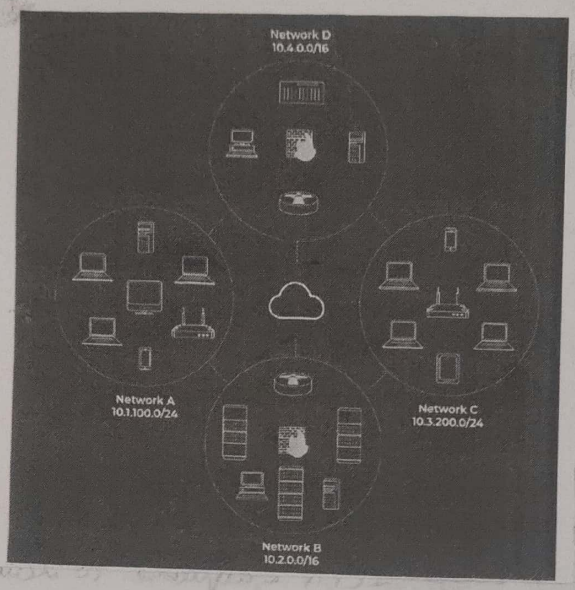
TCP/UDP ping scan: This scan sends packets to TCP ports and UDP ports to determine live hosts.

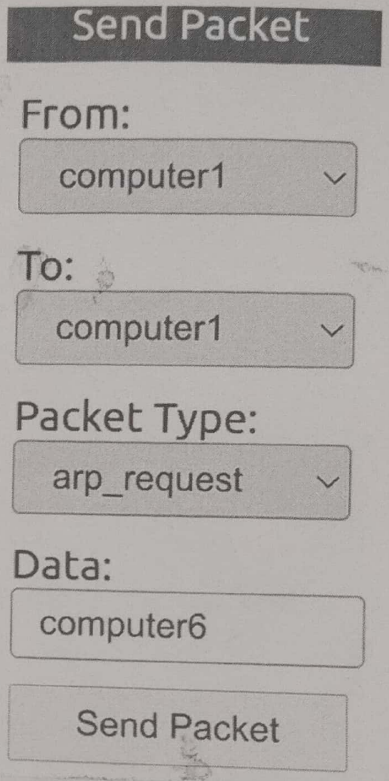


Scanned with OKEN Scanner

Subnetworks:

A network segment is a group of computers, linked through a shared medium, like an ethernet switch or WiFi access point. In IP networks, a subnetwork typically consists of one or more network segments connected together and configured to use the same router. A network segment is a physical connection, while a subnetwork is a logical connection.



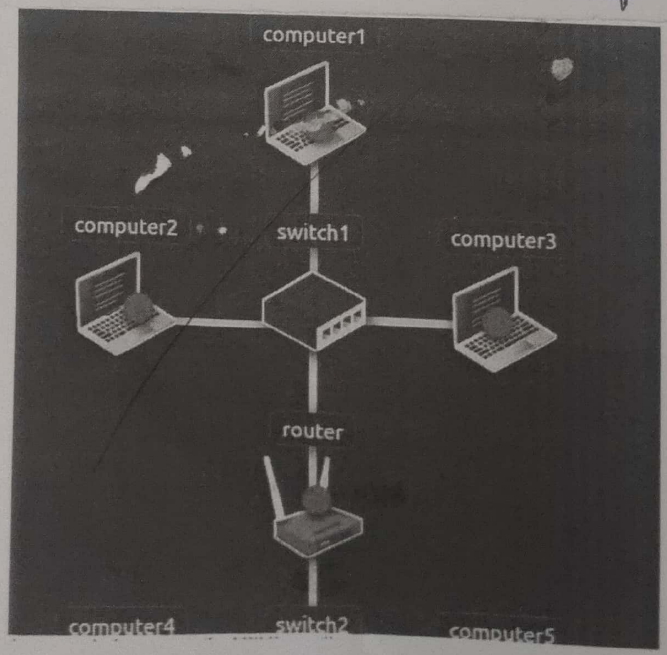


i) How many devices can see the ARP request?

⇒ 4.

ii) Did computer 6 receive the ARP request? (Y/N).

⇒ N.



Scanned with OKEN Scanner

i) How many devices can see the ARP request?

⇒ 4

ii) Did computer 6 reply to the ARP request? (Y/N)

⇒ Y.

Discovering Live hosts:

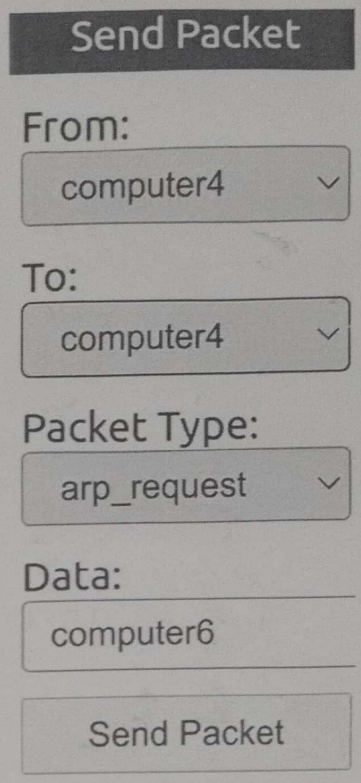
TCP/IP layers:

⇒ ARP from link layer,

⇒ ICMP from Network layer.

⇒ TCP from Transport layer.

⇒ UDP from Transport layer



What is the name of the first device that responded to the first ARP request?

⇒ router,

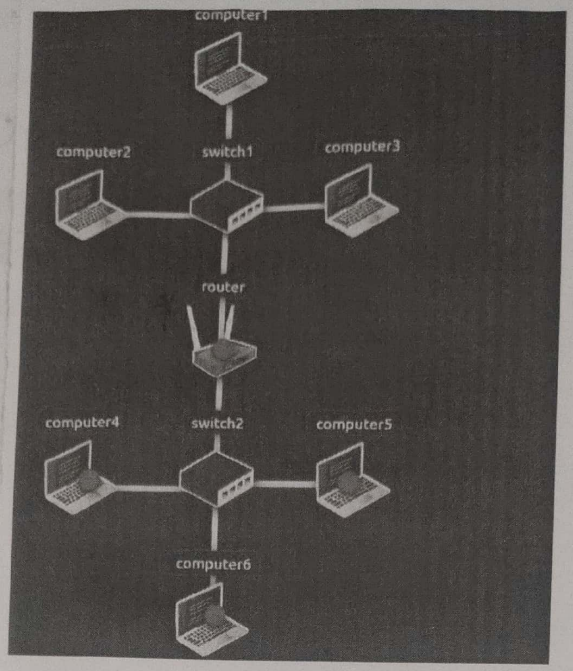
What is the name of the first device that responded to the second ARP request?

⇒ Computer 5,

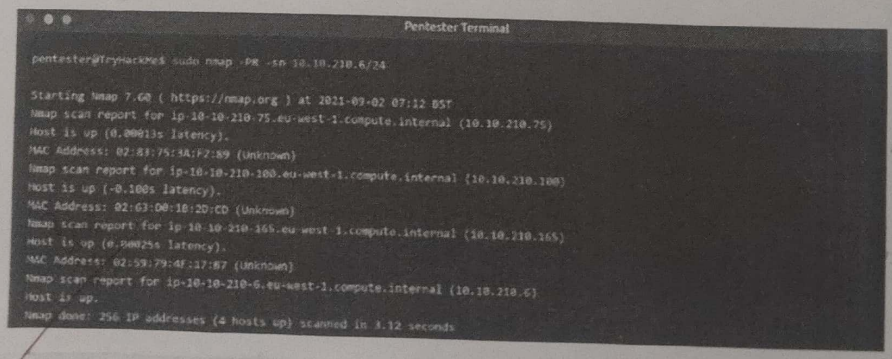
Send another ping request. Did it require new ARP requests? (Y/N).

⇒ N.

Scanned with OKEN Scanner

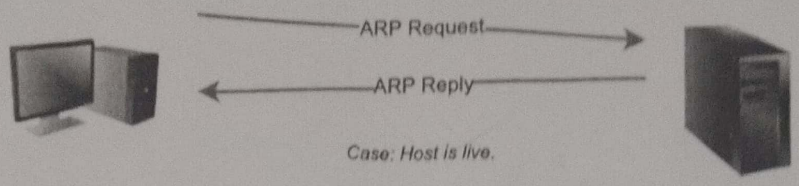


7	Application Layer			
6	Presentation Layer	Application Layer	HTTP, HTTPS, SMTP, POP3, IMAP, SSH, FTP, SNMP, Telnet, RDP,...	
5	Session Layer			
4	Transport Layer	Transport Layer	TCP, UDP	
3	Network Layer	Network Layer	IPv4, IPv6, ICMP, IPsec	
2	Data Link Layer	Link Layer	ARP, Ethernet (802.3), WiFi (802.11), DSL, Bluetooth,	
1	Physical Layer			



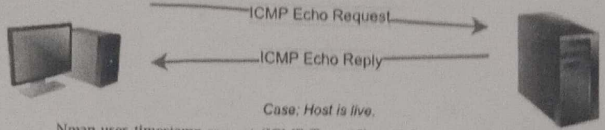
Scanned with OKEN Scanner

nmap -PR -sn TARGET



Case: Host is live.

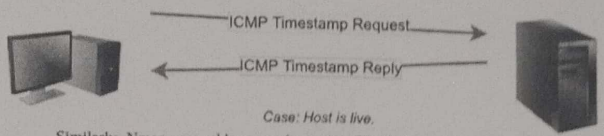
nmap -PE -sn TARGET



Case: Host is live.

Nmap uses timestamp request (ICMP Type 13) and checks whether it will get a Timestamp reply (ICMP Type 14). Adding the -PP option tells Nmap to use ICMP timestamp requests. As shown in the figure below, you expect live hosts to reply.

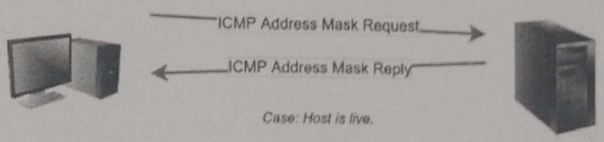
nmap -PP -sn TARGET



Case: Host is live.

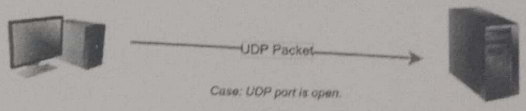
Similarly, Nmap uses address mask queries (ICMP Type 17) and checks whether it gets an address mask reply (ICMP Type 18). This scan can be enabled with the option -PM. As shown in the figure below, live hosts are expected to reply to ICMP address mask requests.

nmap -PM -sn TARGET



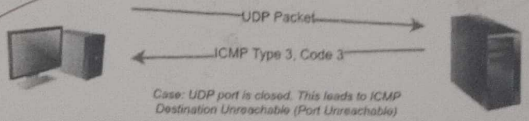
Case: Host is live.

nmap -PU -sn TARGET



Case: UDP port is open.

nmap -PU -sn TARGET



Case: UDP port is closed. This leads to ICMP Destination Unreachable (Port Unreachable)

Answer the questions below

Which TCP ping scan does not require a privileged account?

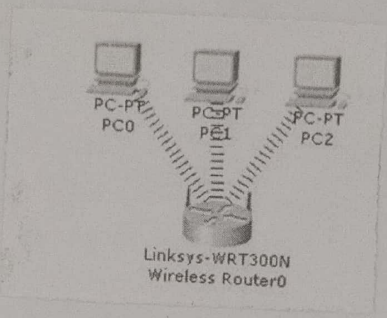
Scanned with OKEN Scanner

Ex.No: 8b Configuration of wireless LAN using CISCO packet Tracer.

Aim:

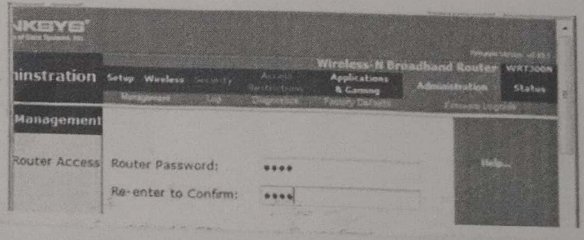
Configuration of wireless LAN using CISCO packet tracer.

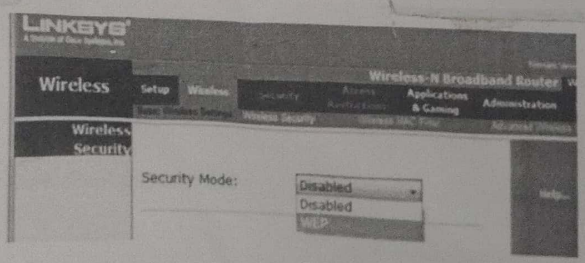
Design a topology with 3 Pc's connected from linksys wireless routers.



Perform following configuration:

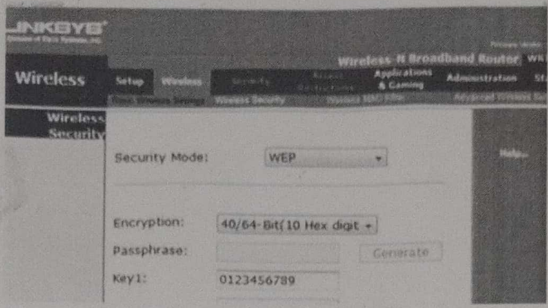
* Configure Static IP on PC and wireless router.
* Set SSID to mother network.
* Set IP address of router to 192.168.0.1, PC0 to 192.168.02, PC1 to 192.168.0.3 and PC2 to 192.168.0.4
* Secure your network by configuring WAP key on router.
* Connect Pc by using WAP key.

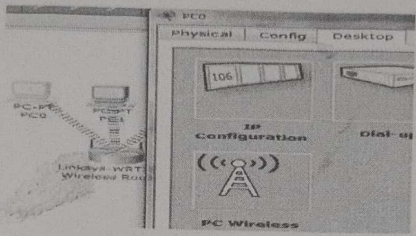




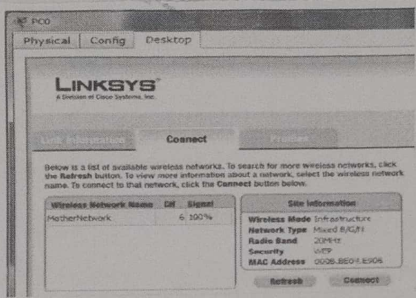
Scanned with OKEN Scanner

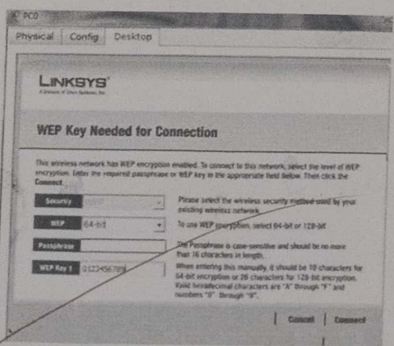
Key1 to 0123456789





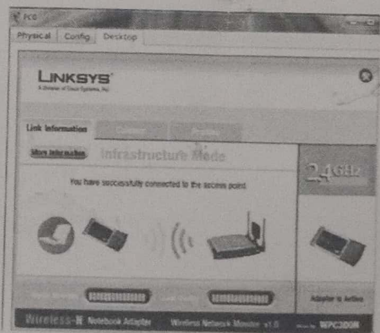
llck on connect tab and click on Refresh button





connect you with wireless router.

u can see in image below that system is connected. And PCI card is



Student Observation:

c) What is SSID of a wireless router?

⇒ It is the name of a wireless network / router used to identify and connect to a Wi-Fi network.

d) What is the security key in a wireless router?

⇒ It is the password used to authenticate and secure a wireless network.

e) Configure a simple wireless LAN using a real access point?

⇒ i) Connect the access point to the computer.

ii) Open the router's configuration page.

iii) Set the SSID.

iv) Select a security method such as WAP2 / WPA3.

v) Set a password.

vi) Save the configuration & restart the router if required.

vii) Test connectivity using ping between the connected PCs.

Scanned with OKEN Scanner

x:NO:9 Implementation of Subnetting in CISCO packet Tracer.

Aim:

Implementation of subnetting in CISCO packet tracer simulator.

Creating a network topology:

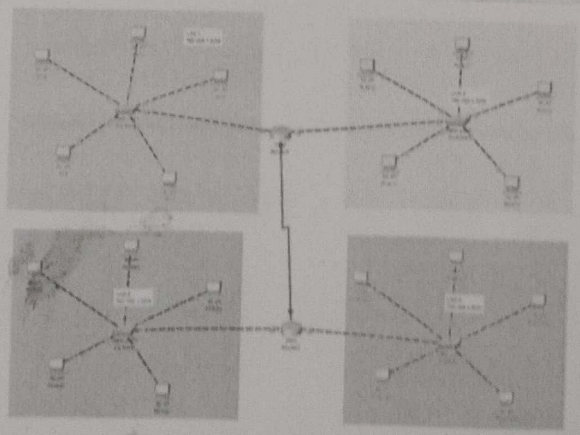
The first step in implementing classless IP subnetting is to create a network topology in packet tracer. To create a network topology in Packet tracer, select the "New" Button in the top left corner, then select "Network" and "generic".

Adding the Devices:

Once we have created our network topology, we can add devices to it. Here, we will be adding routers, switches and PC's. To add a device, select the device from the bottom left corner and drag it onto the network topology.

Subnetting:

To subnet the network address of 192.168.1.0/24 to provide enough space for at least 5 addresses for end devices, the switch and the router, we can use a /27 subnet mask.



The IP addressing for the network shown in the topology can be as follows:

Router R_1

Gigabit Ethernet 0/0: 192.168.1.1

Gigabit Ethernet 0/1: 192.168.2.1

Switch S_1:

Fast ethernet 0/1: 192.168.1.0/27

PC1: 192.168.1.11

PC2: 192.168.1.12

PC3: 192.168.1.13

PC4: 192.168.1.14

PC5: 192.168.1.15

Fast Ethernet 0/2: 192.168.2.0/27

PC1: 192.168.2.11

PC2: 192.168.2.12

PC3: 192.168.2.13

PC4: 192.168.2.14

PC5: 192.168.2.15

Router R_2:

Fast Ethernet 0/0: 192.168.3.1

Fast Ethernet 0/1: 192.168.4.1

Switch S_2:

Fast Ethernet 0/1: 192.168.3.0/27

PC1: 192.168.3.11

PC2: 192.168.3.12

PC3: 192.168.3.13

PC4: 192.168.3.14

PC5: 192.168.3.15

Configuring the devices:

Now that we have added our devices and connected them, we can start configuring them. We will start by configuring the router. Right-click on the router and select "CLI". This will open the command line interface (CLI) for the router.

Scanned with OKEN Scanner

Testing the network:

Now that our network topology is configured, we can test the network; open a command prompt on each PC and try to ping the outer PC. If the ping is successful, then the network is functioning properly.

Student Observation:

a) Write down your understanding of subnetting.

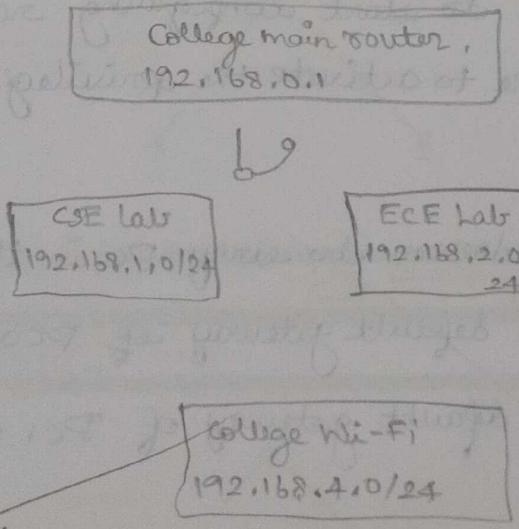
⇒ Subnetting is the process of dividing a large network into smaller logical networks called ~~network~~ subnets.

b) What are the advantages of implementing subnetting within a network?

⇒
* Reduces network traffic.
* Improves Security.
* Provides scalability.
* Simplifies network management.

c) Find out whether subnetting is implemented in your college. If yes, draw and list down the subnets used with IP addresses.

⇒ Yes, subnetting is commonly implemented in college networks to separate departments.



Conclusion:

Subnetting helps a college manage its network efficiently by dividing it into smaller networks.

Scanned with OKEN Scanner

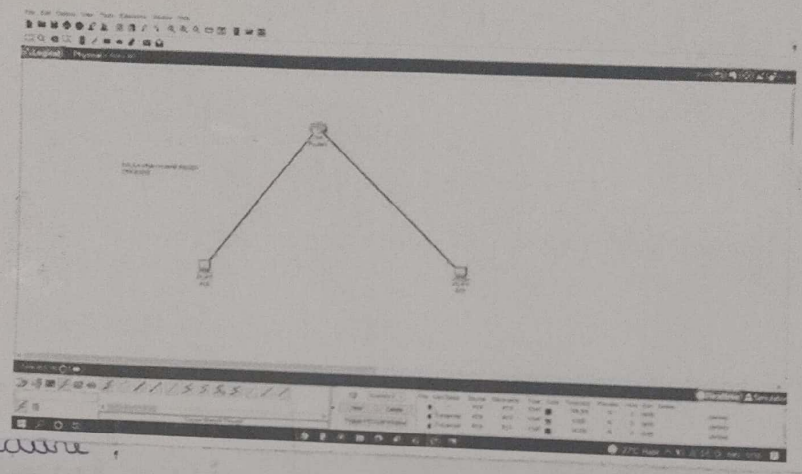
Ex:No:10 Internetworking with routers in CISCO packet Tracer Simulator

a) Aim:

Internetworking with routers in CISCO Packet tracer simulator.

d) Design and configure a simple network using a router.

In this network, a router and 2 PCs are used. Computers are connected with routers using a Copper straight-through cable. After forming the network, to check network connectivity a simple PDU is transferred from PC0 to PC1.



Procedure:

Step 1: (Configuring router 1):

i) Select the router and Open CLI.

ii) Press Enter to start configuring router 1.

iii) Type enable to activate the privileged mode.

Step 2:

i) Assign IP addresses to every PC in the network.

ii) Assign the default gateway of PC0 as 192.168.10.1

iii) Assign the default gateway of PC1 as 192.168.20.1

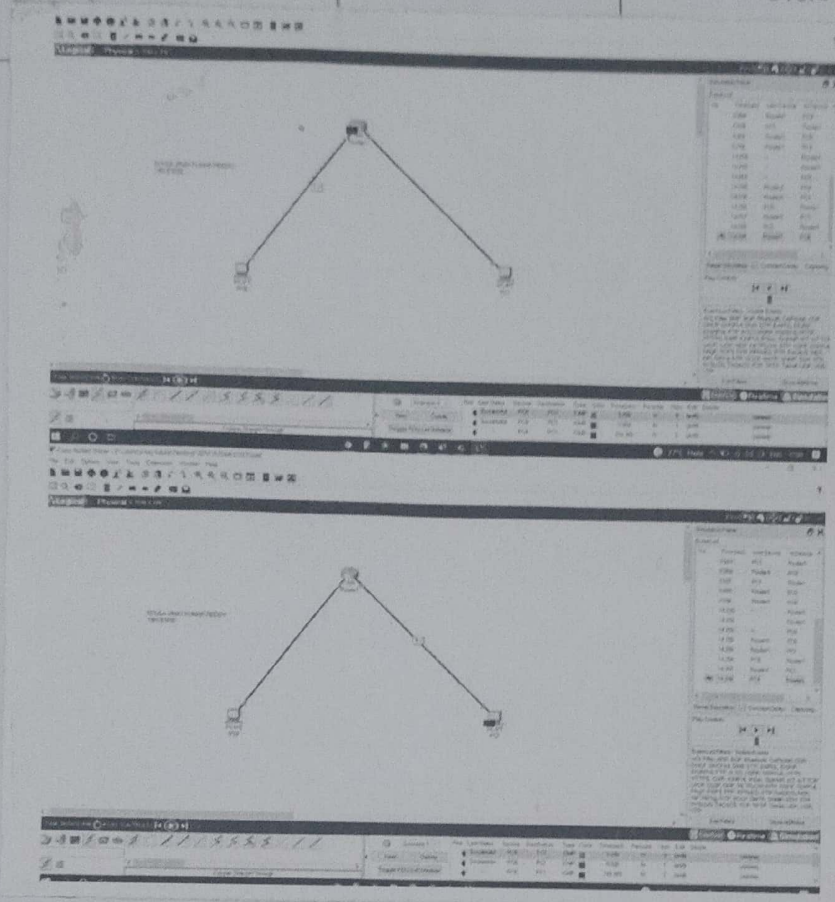
Step 3:

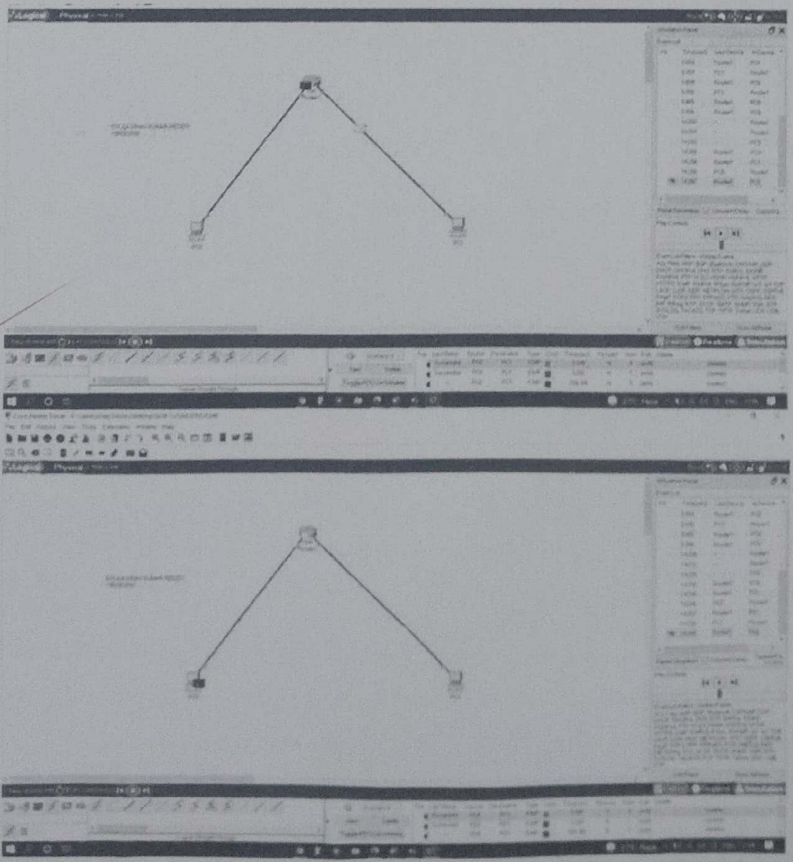
i) Connect fast ethernet 0 port of PC0 with fast ethernet 0/0 port of router 1 using a copper straight through cable.

Scanned with OKEN Scanner

ii) Connect fastethernet 0 port of PC1 with fast ethernet 0/1 port of router1 using a copper straight through cable.

Device name	IP address (0/0)	Subnet mask.	IP address (0/1)	Subnet mask
Router 1	192.168.10.1	255.255.255.0	192.168.20.1	255.255.255.0



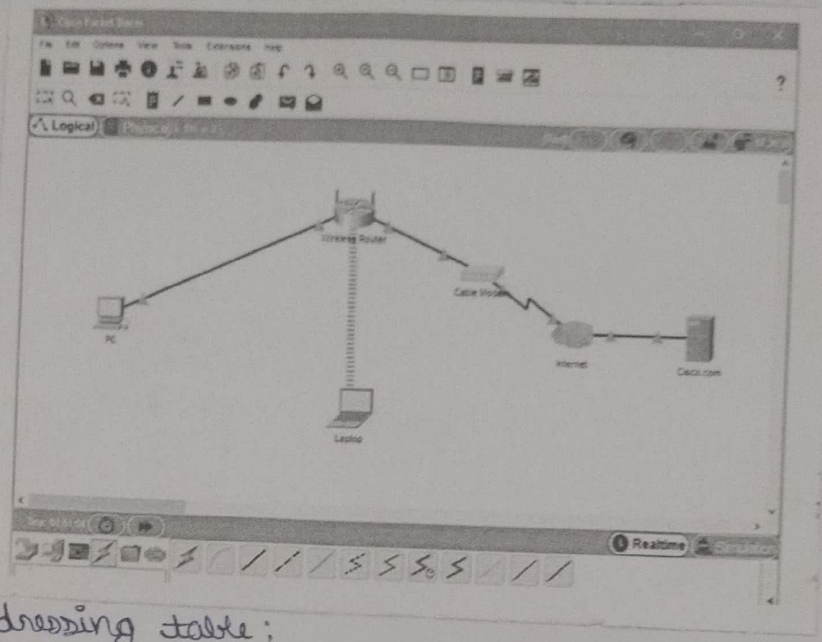


Scanned with OKEN Scanner

Ex.No:10b Design & Configure an internetwork using router, DHCP server.

Aim:

Design & Configure an internetwork using wireless router, DHCP server and internet cloud.



Addressing table:

Device	Interface	IP address	Subnet mask	default gateway
PC	ethernet 0.	DHCP		192,168.0.1
wireless router	Internet	DHCP		
laptop	wireless 0	DHCP.		

Objectives:

i) Build a simple network in the logical topology workspace.

ii) Configure the network devices.

iii) Test Connectivity between network devices.

iv) Save the file and close Packet Tracer.

i)

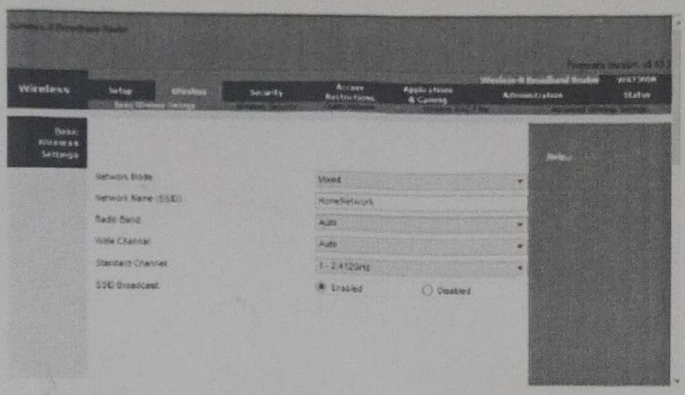
Step 1: Launch Packet Tracer.

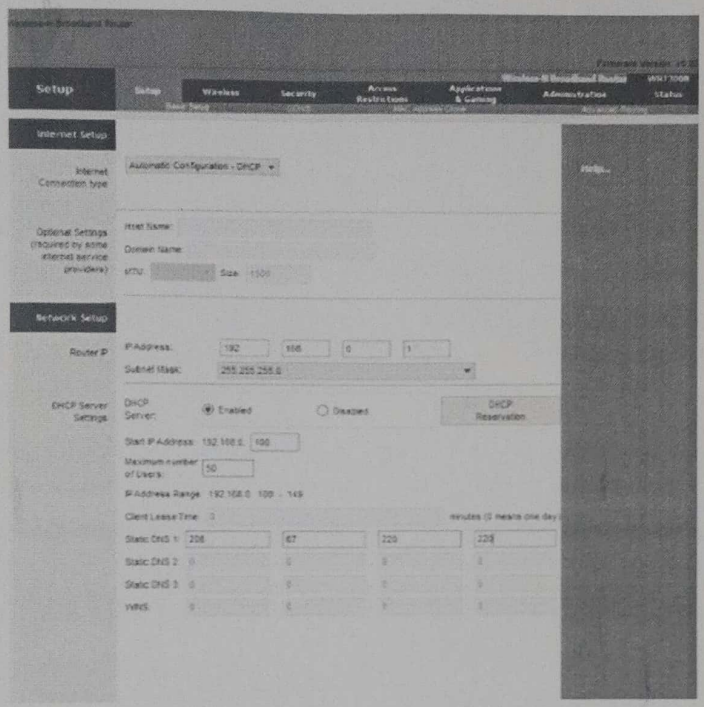
Step 2: Build the topology.

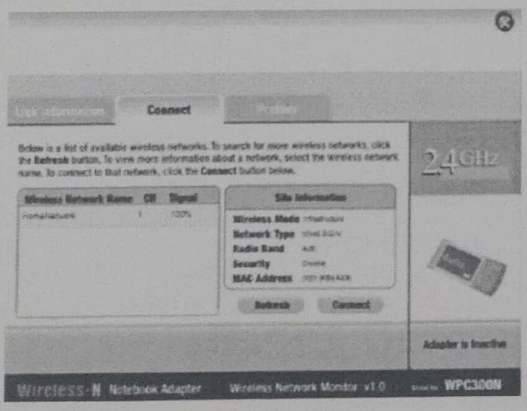
Scanned with OKEN Scanner

ii) Step 1: Configure the wireless router,

Step 2: Configure the laptop.



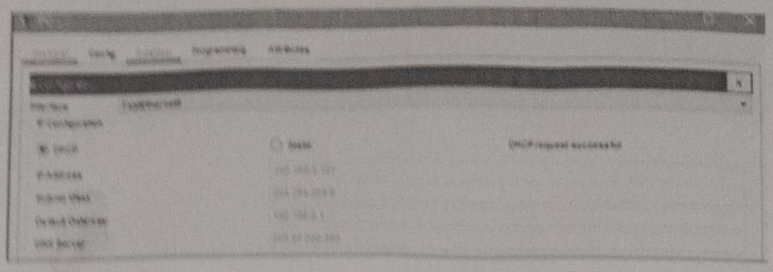




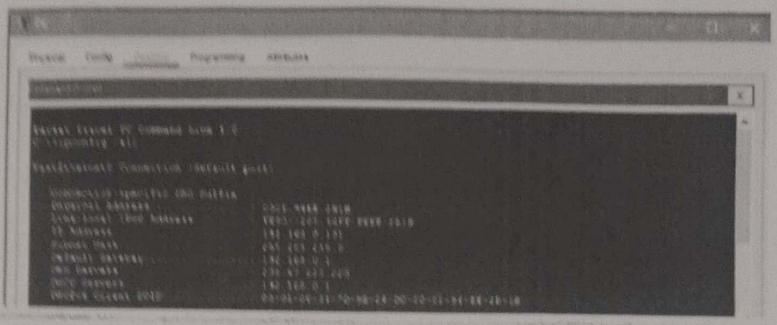
Step 3: Configure the PC,

Click on the PC icon on the packet tracer logical workspace and select the Desktop tab and then the IP configuration icon.

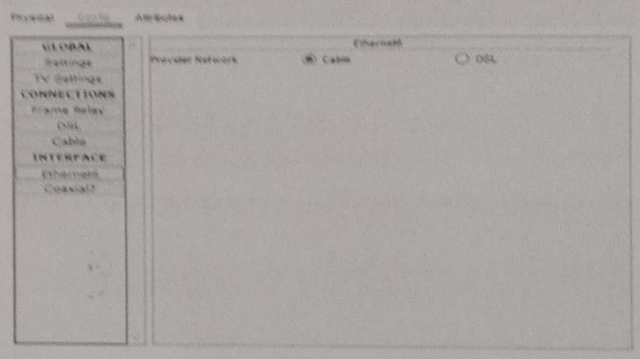
Scanned with OKEN Scanner

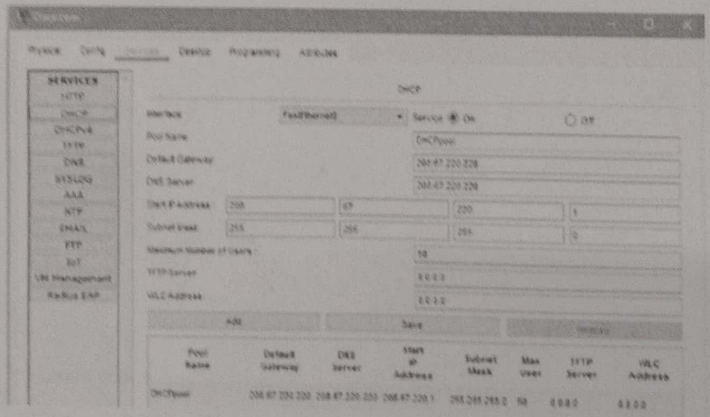


Click on the Command Prompt icon. Verify that the PC has received an IPv4 address by issuing the **ipconfig /all** command from the command prompt as shown in the figure. The PC should receive an IPv4 address in the 192.168.0.x range.



Step 4: Configure the Internet cloud.

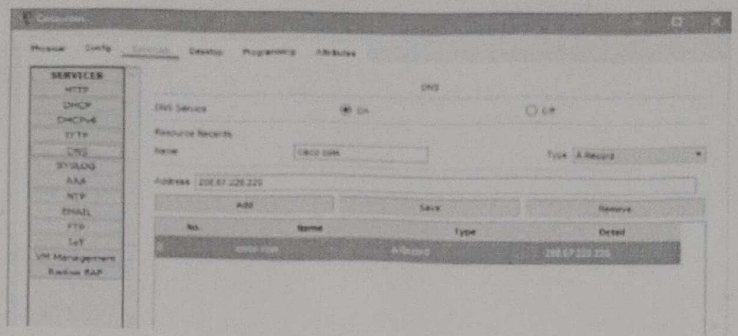


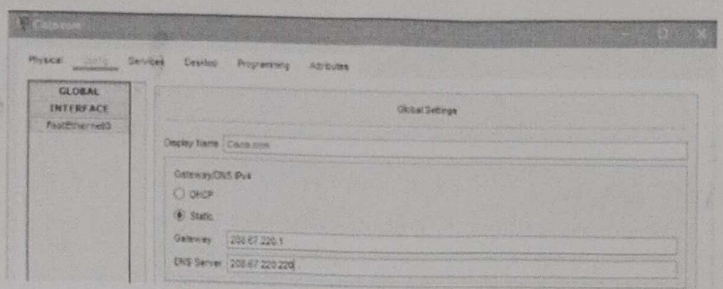


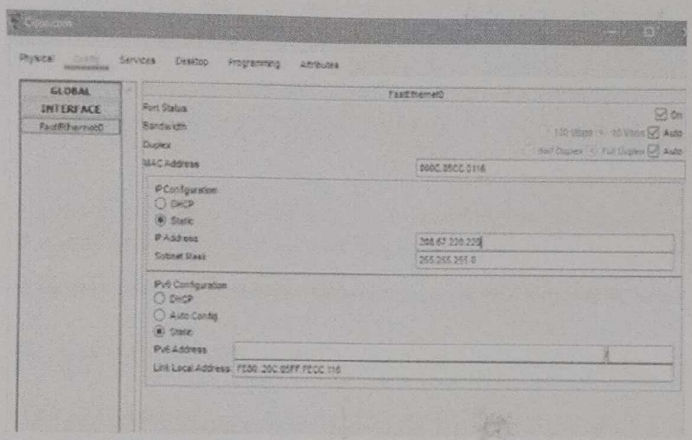
Step 5: Configure the Cisco.com server.

click add to add the pool.

Scanned with OKEN Scanner



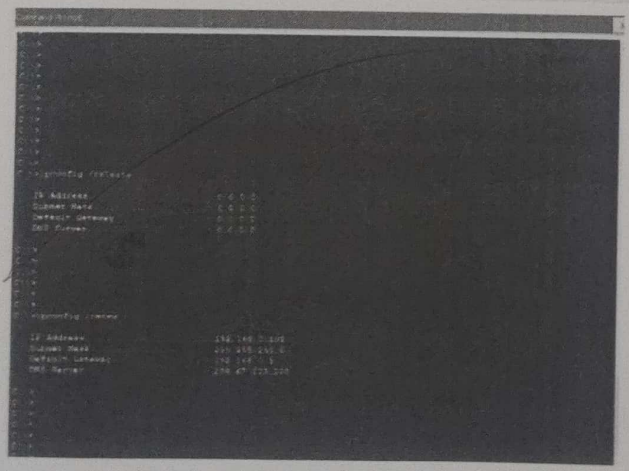




ii) Step 1:

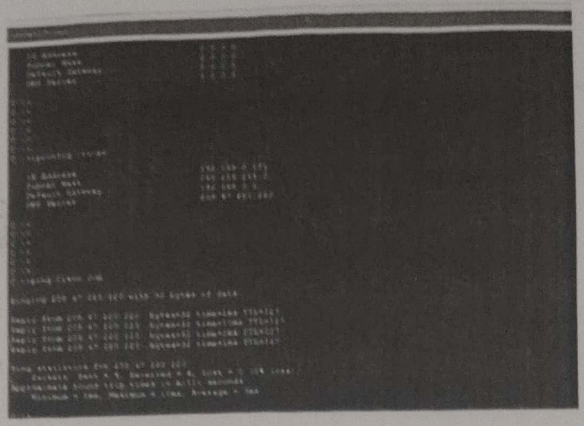
Refresh the IPV4 settings on the PC.

a) Verify that the PC is receiving IPV4 configuration information from DHCP.



b) Test connectivity to the Cisco.com server from the PC.

Scanned with OKEN Scanner



Student Observation;

1) write down the key features of configuring wireless router and DHCP server.

⇒ Wireless router;

* Provides wireless network connectivity.
* Configures SSID.

DHCP server;

* Assigns DNS server addresses.
* Avoids IP address conflicts.

2) What is the significance of DHCP server in inter-networking?

⇒ * Automatically assigns IP addresses.
* Reduces manual configuration.
* Simplifies network administration.
* Prevents duplicate IP address conflicts.

Scanned with OKEN Scanner

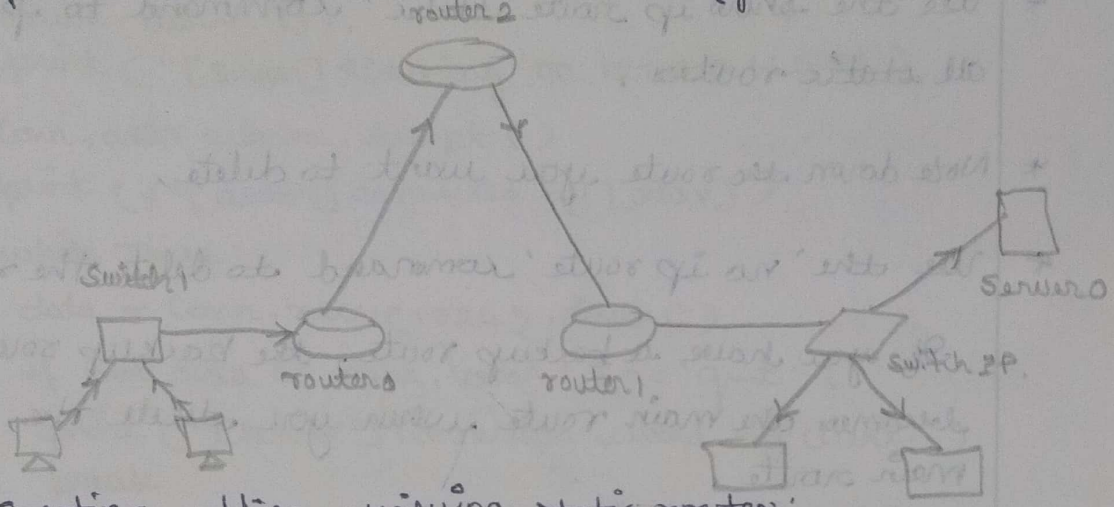
Ex:NO:11 Simulate Static routing Configuration using CISCO packet Tracer.

Aim:

Simulate static routing Configuration using CISCO packet tracer.

Setting up a lab:

Create a packet tracer lab as shown in the following image or download the following and load it on packet tracer with initial IP configuration.



Creating, adding, verifying static routes:

Routers automatically learn their Connected networks, we only need to add routes for the networks that are not available on the router's interfaces, for example, network 10.0.0.0/8, 20.0.0.0/8 and 40.0.0.0/8 are directly connected to router().

Router 1 requirements:

* Create two routes for network 10.0.0.0/8 and Configure the first route as the main route and the second route as a backup route.
* Verify the router adds only main routes to the routing table.

Router 2 requirements:

Create static routes for network 10.0.0.0/8 and network 30.0.0.0/8 and verify the router adds both

Scanned with OKEN Scanner

routes to the routing table.

Verifying static routing:

On router(), we configured two routes for network 30.0.0.0/8. These routes are via router 1 and via router 2. We set the first route as the main route and the second route as the backup route.

Deleting a static route:

To delete a static route, use the following steps.

* Use the 'show ip route static' command to print all static routes.
* Note down the route you want to delete.
* Use the 'no ip route' command to delete the route.

If you have a backup route, the backup route becomes the main route when you delete the main route.

Router() (config)# router rip
router() (config-router)# network.
10.0.0.0
router() (config-router)# network 192.168.1.252.

⇒ router rip command tell router to enable the RIP routing protocol.

Network command allows us to specify the networks which we want to advertise.

Scanned with OKEN Scanner

Ex.No:12a) 12. End - end communication at Transport layer.

Aim:

Implement echo client using TCP/UDP sockets.

Algorithm:

```
import socket
import threading
def start_server():
  server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('127.0.0.1', 5000))
server.listen(1)
print("[server] listening on 127.0.0.1:5000")
conn, addr = server.accept()
print(f"[server] connected by {addr}")
while True:
  data = conn.recv(1024).decode()
  if not data or data.lower() == 'quit':
   print("[Server] Connection closed by client.")
   break
   print(f"[Server] received: {data}")
   conn.send(f"Server received: {data}".encode())
conn.close()
server.close()
if __name__ == "__main__":
server_thread = threading.thread(target = start_
            server, daemon = True)
 server_thread.start()
 start_client().
```

Output:

[server] listening on 127.0.0.1:5000 [client] connected to server

[server] connected by ('127.0.0.1', 53458) Enter message (or 'quit' to exit):

hello.

[Server] Received: hello.

[Server Response] Server received: hello.

Ex.No:12b) End-end communication at Transport layer.

Aim:

Implement chat client server using TCP/UDP packet sockets.

Algorithm:

```
import socket
import threading
def udp_server():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM
    s.bind(('127.0.0.1', 12000))
    print("[Server] running on 127.0.0.1:12000")
    while True:
    data, addr = s.recvfrom(1024)
    msg = data.decode()
    if msg.lower() == 'quit':
    print(f"[Server] client {addr} ended
                the connection.")
     break
     print(f"[Server] received from {addr}: {msg}")
     s.sendto(data, addr)
     s.close()
     print("[Server] stopped.")
def udp_client():
    c = socket.socket(socket.AF_INET,
                socket.SOCK_DGRAM)
    server = ('127.0.0.1', 12000)
    while TRUE:
    msg = input("Enter message (or 'quit' to stop):")
    c.sendto(msg.encode(), server)
    if msg.lower() == 'quit':
        print("[CLIENT] closing connection.....")break
    data, _ = c.recvfrom(1024)
```

Scanned with OKEN Scanner

```
s.close()
print("[Server] Stopped.")
def udp-client():
 c=socket.socket(socket,AF-INET, saket, sock-DGRAM)
 server=('127.0.0.1', 12000)
 while True:
  msg=input("Enter message (or 'quit' to stop):")
  c.sendto(msg.encode(), server)
  if msg.lower()=='quit':
   print("[client] closing connection....") break
  data,-=c.recvfrom(1024)
  print("Echo from Server:", data.decode())
  c.close()
  print("[client] Stopped.")
  if -name-=="-main-":
   threading.thread(target=udp-server,
                    daemon=True).start()
   udp-client().
```

Output:

Enter message (or 'quit' to stop): [Server] running on 127.0.0.1:12000 hi.

[Server] Received from ('127.0.0.1', 49670): hi

Echo from Server: hi

Enter message (or 'quit' to stop): hello

[Server] received from ('127.0.0.1', 49670): hello

Echo from server: hello.

Scanned with OKEN Scanner

EX.NO:13 Implement your own ping program.

Aim:

Implement your own ping program.

Algorithm:

```
import os
import platform
import subprocess
def ping(host="google.com", count=4):
param="-n" if platform.system().lower()==
        "windows" else "-c"
command=["ping", param, str(count), host]
try:
output=subprocess.check_output(command, universal_
                               newlines=True)
print(output)
except Exception as e:
print(f"Ping failed: {e}")
if __name__=="__main__":
ping("8.8.8.8")
```

Output:

Pinging 8.8.8.8 with 32 bytes of data:

reply from 8.8.8.8: bytes=32 time=6ms TTL=119

reply from 8.8.8.8: bytes=32 time=9ms TTL=119.

Ping statistics for 8.8.8.8:

Packets: sent=4, received=4, lost=0 (0% loss)

Approximate round trip times in milli-seconds:

Minimum=5ms, Maximum=9ms, Average=7ms.

Scanned with OKEN Scanner

EX.NO:14 RAW sockets to implement packet sniffing.

Aim:

to determine the RAW sockets to implement packet sniffing.

Code:

```
from scapy.all import sniff
from scapy.layers.inet import IP,TCP,UDP,ICMP
def packet_callback(packet):
    if IP in packet:
        ip_layer = packet[IP]
        protocol = ip_layer.proto
        src_ip = ip_layer.src
        dst_ip = ip_layer.dst
        protocol_name = ""
        if protocol == 1:
            protocol_name = "ICMP"
        elif protocol == 6:
            protocol_name = "TCP"
        elif protocol == 17:
            protocol_name = "UDP"
        else:
            protocol_name = "Unknown Protocol".
        print(f"Protocol : {protocol_name}")
        print(f"Source IP: {src_ip}")
        print(f"Destination IP: {dst_ip}")
        print("-",*50)
    sniff(iface = 'Wi-Fi', prn = packet_callback,
                    filter = "ip", store = 0).
```

Output:

Protocol : TCP

Source IP : 57.155.141.17

Destination IP : 192.168.1.6

Protocol : UDP

Source IP : 192.168.1.6

Destination IP : 192.168.1.1

Completed

13/10/25